

گزارش سوال ۳ عملی - تمرین ۵ - سیستم عامل

گام ۱. رابط کاربری cli را در تمرین قبل پیاده‌سازی کردیم و عملکرد آن را نیز در گزارش آن بررسی کردیم. گزارش تمرین قبل نیز در همین پوشه موجود است.

گام ۲. freelist خواسته شده را به صورت زیر طراحی کرده و در قسمت‌های مختلف کد از آن استفاده می‌کنیم که جزئیات دقیق نحوه پیاده‌سازی را می‌توانید در فایل‌های کد پیوست مطالعه کنید.

```
// ----- HW5 -----  
// ----- FreeList -----  
#define FREELIST_ENTRY_SIZE sizeof(free_entry_t)  
  
typedef struct free_entry {  
    uint32_t start;    // start offset of free region  
    uint32_t end;      // end offset (exclusive)  
    uint32_t next;     // next free block in the list  
} free_entry_t;
```

پیاده‌سازی بالا دقیقاً مطابق، خواسته دستور کار است. همچنین از آنجا که مقدار next هر free_entry برابر است با مقدار start در free_entry بعدی، می‌توان برای ساده‌سازی بیشتر، متغیر start را از struct free_entry حذف کرده.

گام ۳. تابع alloc_data_block را که از قبل داشتیم حذف کرده و آن را با تابع alloc(size) جایگزین می‌کنیم:

```
uint32_t fs_alloc(uint32_t size) {  
    if (size == 0) return 0;  
  
    superblock_t sb;  
    load_superblock(&sb);  
  
    uint32_t prev_ptr = 0;  
    uint32_t curr_ptr = sb.free_block_head;  
    free_entry_t curr;  
  
    while (curr_ptr != 0) {  
        load_free_entry(curr_ptr, &curr);  
  
        uint32_t available = curr.end - curr.start;  
  
        if (available >= size) {  
            // Found a spot  
            uint32_t alloc_start = curr.start;
```

همچنین تابع `free(start, size)` خواسته شده را اضافه می‌کنیم:

```
void fs_free(uint32_t start, uint32_t end) {
    if (start >= end) return;

    superblock_t sb;
    load_superblock(&sb);

    // 1. Create the new entry in memory
    free_entry_t new_ent;
    new_ent.start = start;
    new_ent.end = end;
    new_ent.next = 0;

    // 2. Find insertion spot (sorted by start address)
    uint32_t prev_ptr = 0;
    uint32_t curr_ptr = sb.free_block_head;
    free_entry_t curr;

    while (curr_ptr != 0) {
```

گام ۴. از تابع `alloc` هنگام ایجاد فایل (و درون توابع دیگر):

```
uint32_t create_file(const char *name) {
    if (strlen(name) >= FILENAME_MAX_LEN) {
        printf("Error: filename too long.\n");
        return 0;
    }

    // Check if file already exists
    if (find_file(name, NULL) != 0) {
        printf("Error: file already exists.\n");
        return 0;
    }

    uint32_t off = fs_alloc(sizeof(file_entry_t));
    if (off == 0) return 0;

    file_entry_t ent;
    memset(&ent, 0, sizeof(ent));
```

و از تابع free برای دستورات rm و shrink استفاده می‌کنیم:

```
void shrink_file(uint32_t newsize) {
    if (g_open_file_offset == 0) return;

    file_entry_t ent;
    load_file_entry(g_open_file_offset, &ent);

    if (newsize >= ent.size) return;

    uint32_t old_end = ent.first_block + ent.size;
    uint32_t new_end = ent.first_block + newsize;

    // Return the difference to the free list
    fs_free(new_end, old_end);

    ent.size = newsize;
    save_file_entry(g_open_file_offset, &ent);
    printf("File shrunk.\n");
}
```

```
void remove_file() {
    if (g_open_file_offset == 0) return;

    file_entry_t ent;
    load_file_entry(g_open_file_offset, &ent);

    // 1. Unlink from file list
    superblock_t sb;
    load_superblock(&sb);
    uint32_t target = g_open_file_offset;
    if (sb.file_list_head == target) { ...
    } else { ...
    }
    sb.file_count--;
    save_superblock(&sb);

    // 2. Free Data
    if (ent.first_block != 0 && ent.size > 0) {
        fs_free(ent.first_block, ent.first_block + ent.size);
    }

    // 3. Free Metadata entry itself
    // We treat the file_entry slot as a small memory block and free it too!
    fs_free(g_open_file_offset, g_open_file_offset + sizeof(file_entry_t));

    g_open_file_offset = 0;
}
```

گام ۵. در این گام دستور viz را پیاده سازی کرده و به کمک آن صحت عملکرد freelist را بررسی می کنیم.

در ابتدای ایجاد filesys.db جدید، تنها یک فضای خالی بزرگ داریم که از بعد از superblock شروع شده و تا انتهای filesys.db ادامه دارد:

```
o martin@Martin:~/Term5/OS/HW/4$ ./myfs
FS CLI - type 'help' for commands.
myfs> init filesys.db
Created new filesystem 'filesys.db'
myfs> viz

----- Free Space Visualization -----
Total free blocks: 1

Region 0: start =      4096 end = 268435456 size = 268431360 bytes
-----

myfs> █
```

سپس چندین فایل به طور متوالی ایجاد کرده و کاهش این فضا را مشاهده می کنیم:

```
myfs> open f1 1
File created and opened: f1
myfs> write 1000000 1
a
Wrote 1 bytes.
myfs> myfs> open f2 1
File created and opened: f2
myfs> write 2000000 1
b
Wrote 1 bytes.
myfs> myfs> open f3 1
File created and opened: f3
myfs> write 3000000 1
c
Wrote 1 bytes.
myfs> myfs> open f4 1
File created and opened: f4
myfs> write 4000000 1
d
Wrote 1 bytes.
myfs> myfs> viz

----- Free Space Visualization -----
Total free blocks: 1

Region 0: start = 10004436 end = 268435456 size = 258431020 bytes
-----

myfs> █
```

مشاهده می شود که ۴ فایل متوالی ایجاد کردیم و اکنون فضای خالی از حدود آدرس ۱۰ مگابایت شروع می شود.

در ادامه فایل ۲ که حدود ۲ مگابایت حجم داشت را حذف می‌کنیم:

```
myfs> open f2 0
File opened: f2 (size = 2000001 bytes)
myfs> rm
File removed.
myfs> viz

----- Free Space Visualization -----
Total free blocks: 2

Region 0: start = 1004181 end = 3004266 size = 2000085 bytes
Region 1: start = 10004436 end = 268435456 size = 258431020 bytes
-----

myfs> █
```

فضای خالی پس از حذف آن به درستی تشخیص داده می‌شود.

اکنون فایل ۳ را shrink می‌کنیم و ۳ فضای خالی خواهیم داشت:

```
myfs> open f3 0
File opened: f3 (size = 3000001 bytes)
myfs> shrink 1500000
File shrunk.
myfs> viz

----- Free Space Visualization -----
Total free blocks: 3

Region 0: start = 1004181 end = 3004266 size = 2000085 bytes
Region 1: start = 4504350 end = 6004351 size = 1500001 bytes
Region 2: start = 10004436 end = 268435456 size = 258431020 bytes
-----

myfs> █
```

حال همانطور که انتظار می‌رود، پس از حذف کلی فایل ۳، فضای خالی قبل و بعد از آن به هم متصل شده و به عنوان یک

فضای خالی شناسایی خواهند شد:

```
myfs> rm
File removed.
myfs> viz

----- Free Space Visualization -----
Total free blocks: 2

Region 0: start = 1004181 end = 6004351 size = 5000170 bytes
Region 1: start = 10004436 end = 268435456 size = 258431020 bytes
-----

myfs> █
```